

SIMPLY FPU

by **Raymond Filiatreault**
Copyright 2003

Chap. 7 Comparison instructions

The FPU instructions covered in this chapter perform no mathematical operation on numerical data. Their main purpose is simply to compare the value in the FPU's top data register ST(0) to the value in another FPU data register, to floating point values in memory, to integer values in memory, to +0.0, or to examine its content.

The comparison instructions covered in this document are (in alphabetical order):

FCOM	COMpare ST(0) to a floating point value
FCOMI	COMpare ST(0) to ST(<i>i</i>) and set CPU flags
FCOMIP	COMpare ST(0) to ST(<i>i</i>) and set CPU flags and Pop ST(0)
FCOMP	COMpare ST(0) to a floating point value and Pop ST(0)
FCOMPP	COMpare ST(0) to ST(1) and Pop both registers
FICOM	COMpare ST(0) to an integer value
FICOMP	COMpare ST(0) to an integer value and Pop ST(0)
FTST	TeST ST(0) by comparing it to +0.0
FUCOM	Unordered COMpare ST(0) to a floating point value
FUCOMI	Unordered COMpare ST(0) to ST(<i>i</i>) and set CPU flags
FUCOMIP	Unordered COMpare ST(0) to ST(<i>i</i>) and set CPU flags and Pop ST(0)
FUCOMP	Unordered COMpare ST(0) to a floating point value and Pop ST(0)
FUCOMPP	Unordered COMpare ST(0) to ST(1) and Pop both registers
FXAM	eXAMine the content of ST(0)

FCOM (Compare ST(0) to a floating point value)

Syntax: **fcom** (no operand)
 fcom Src

Exception flags: Stack Fault, Invalid operation, Denormalized value

This instruction performs a signed comparison between the value in the TOP data register ST(0) and the floating point value from the specified source (*Src*). The source can be one of the FPU's data registers or the memory address of a REAL4 or REAL8 value (see Chap.2 for [addressing modes of real numbers](#)). If no operand is specified, the value in ST(0) is compared to the value in ST(1). (The [**FICOM**](#) instruction must be used to compare the value in ST(0) to an integer in memory).

Note that ST(0) cannot be compared to a REAL10 in memory. If that becomes necessary, the REAL10 value must first be loaded to the FPU.

The values of ST(0) and of the source are not modified by this instruction.

The result of the comparison is reported in the condition codes field of the [Status Word](#) as follows (the C1 bit is not used and the C2 bit was not used in early FPUs):

	C3	C2	C0
If ST(0) > source	0	0	0
If ST(0) < source	0	0	1
If ST(0) = source	1	0	0
If ST(0) ? source	1	1	1

An **Invalid** operation exception is detected if ST(0) is empty, or if a data register specified as the source is empty, or if one of the two values is a [NAN](#), setting the related flag in the Status Word. The result would then be indeterminate and return C3=1, C2=1 and C0=1 as indicated above. (Values of [INFINITY](#) will be treated as valid operands and yield a valid result without any exception being detected.)

A **Stack Fault** exception is also detected if ST(0) (or a data register specified as the source) is empty, setting the related flag in the Status Word.

When either of the two values being compared is a [denormalized](#) number, a **Denormal** exception is detected, setting the related flag in the Status Word. The comparison would still yield a valid result.

The signed comparison means that positive values are larger than negative values, including the values of INFINITY which can be positive or negative. The lowest of all valid values would thus be -INFINITY and the greatest would be +INFINITY when used in comparisons.

When the source is a [REAL4](#) or [REAL8](#) in memory, it is first converted to the [REAL10](#) format before the comparison. Because of the rounding being performed before storing REAL4 and REAL8 values in memory, comparing ST(0) to its stored value may rarely yield the "=" result (see the latter part of the [FLD](#) instruction for further explanation). If a computed value must be stored and used for subsequent comparisons, it should be stored as a REAL10 for maximum accuracy.

When the Status Word containing the result of the comparison is stored in AX, the C3, C2 and C0 bits can be transferred to the CPU's flag register (using the [sahf](#) instruction) as the ZF, PF and CF flags respectively. This makes it easier for branching based on the result. Following would be typical code after a comparison with a memory variable.

```
fcom real8_var    ;compare ST(0) with the value of the real8_var variable
fstsw ax          ;copy the Status Word containing the result to AX
fwait             ;insure the previous instruction is completed
sahf              ;transfer the condition codes to the CPU's flag register
jpe error_handler ;the comparison was indeterminate
                  ;this condition should be verified first
                  ;then only two of the next three conditional jumps
                  ;should become necessary, in whatever order is preferred,
                  ;the third jump being replaced by code to handle that case
ja    st0_greater ;when all flags are 0
jb    st0_lower   ;only the C0 bit (CF flag) would be set if no error
jz    both_equal  ;only the C3 bit (ZF flag) would be set if no error
```

FCOMP (Compare ST(0) to a floating point value and POP ST(0))

Syntax: **fcomp** (no operand)
 fcomp Src

Exception flags: Stack Fault, Invalid operation, Denormalized value

This instruction is the same as the FCOM instruction with the only difference that the top data register is popped after the comparison is completed. This instruction is used when the value in ST(0) would no longer be needed for further computation after the comparison has been performed. See the [FCOM](#) instruction for other details. (The [FICOMP](#) instruction must be used to compare the value in ST(0) to an integer in memory and POP the TOP data register).

The popped value is not stored anywhere. The FPU simply modifies the tag of the current TOP register to "empty" in the [Tag Word](#), and increments the TOP field in the [Status Word](#).

An example of using the FCOMP instruction would be when a temporary value has been stored as a REAL10 for later use in comparisons. Memory values in that format are not available for direct comparison like the REAL4 and REAL8 values and must be loaded to the FPU. If a comparison between that temporary value and the value in ST(2) would be required,

```
fld temp_var ;load the REAL10 value of temp_var
               ;temp_var now located in ST(0)
fcomp st(3)   ;compare it to the value in ST(3)
               ;which was formerly ST(2) before loading temp_var
               ;and then discard the temp_var value from the FPU and
               ;re-establish the previous order of values in the FPU registers
fstsw ax      ;retrieve result and proceed as required
               ;(see the FCOM example code)
```

FCOMPP (Compare ST(0) to ST(1) and POP both registers)

Syntax: **fcompp** (no operand)

Exception flags: Stack Fault, Invalid operation, Denormalized value

This instruction is the same as the [FCOM](#) and [FCOMP](#) instructions with the following two differences:

- ST(0) can only be compared to ST(1) and
- both ST(0) and ST(1) are popped after the comparison is completed.

This instruction is used when both the values in ST(0) and in ST(1) would no longer be needed in FPU registers for further computation.

An example of using the FCOMPP instruction would be when a recursive computation is performed until the difference between the current value and the previous one must meet a given criteria. The following typical code assumes global variables for the criteria and for the temporary storage of previous values. It also assumes that the current value is in ST(0).

.data

```
criteria    dt 3.3333e-15 ;could be initialized or set by the program
temp_var    dt ?          ;could be initialized or set by the program
```

.code

```
fld    temp_var    ;load previous value
               ;=> ST(0)=previous value, ST(1)=current value
fsub    st,st(1)    ;difference with current value
fabs     ;get the absolute value of the difference
fld    criteria     ;load the criteria
               ;=> ST(0)=criteria, ST(1)=abs(difference), ST(2)=current value
fcompp   ;compare the criteria to the difference
               ;and discard both values from the FPU
               ;=> ST(0)=current value
fstsw ax    ;retrieve comparison result in the AX register
fwait      ;insure the previous instruction is completed
sahf       ;transfer the condition codes to the CPU's flag register
```

;In this type of code, the computed values should already have been verified to be valid numbers. Their difference should thus be a valid number, as well as the criteria. Therefore no need to check for an indeterminate comparison.

```
ja    criteria_greater ;criteria was ST(0) for comparison
jb    criteria_lower
jz    criteria_equal
```

FUCOM (Unordered Compare ST(0) to a floating point value)

Syntax: **fucom** (no operand)
 fucom Src

Exception flags: Stack Fault, Invalid operation, Denormalized value

This instruction is identical to the FCOM instruction with the only difference that the Invalid operation exception will be detected only if one or both operands are an SNAN type of [NAN](#). The QNANs would not result in an invalid operation but would still set the C0, C1 and C3 condition bits of the [Status Word](#) to 1 (undefined comparison). See the [FCOM](#) instruction for other details.

This specialized instruction could be of use for advanced programming only if SNANs and QNANs features were implemented. A similar advanced instruction is not available for comparing ST(0) with integer values.

FUCOMP (Unordered Compare ST(0) to a floating point value and POP ST(0))

Syntax: **fucomp** (no operand)
 fucomp Src

Exception flags: Invalid operation, Denormalized value

This instruction is identical to the FCOMP instruction with the only difference that the Invalid operation exception will be detected only if one or both operands are an SNAN type of [NAN](#). The QNANs would not result in an invalid operation but would still set the C0, C1 and C3 condition bits of the [Status Word](#) to 1 (undefined comparison). See the [FCOMP](#) instruction for other details.

This specialized instruction could be of use for advanced programming only if SNANs and QNANs features were implemented. A similar advanced instruction is not available for comparing ST(0) with integer values.

FUCOMPP (Unordered Compare ST(0) to ST(1) and POP both registers)

Syntax: **fucompp** (no operand)

Exception flags: Stack Fault, Invalid operation, Denormalized value

This instruction is identical to the FCOMPP instruction with the only difference that the Invalid operation exception will be detected only if one or both operands are an SNAN type of [NAN](#). The QNANs would not result in an invalid operation but would still set the C0, C1 and C3 condition bits of the [Status Word](#) to 1 (undefined comparison). See the [FCOMPP](#) instruction for other details.

This specialized instruction could be of use for advanced programming only if SNANs and QNANs features were implemented.

FCOMI (Compare ST(0) to ST(*i*) and set CPU flags)

Syntax: **fcomi st,st(*i*)**

Exception flags: Stack Fault, Invalid operation

Encoding: DB F0+i

Note: This instruction is valid only for the Pentium Pro and subsequent processors. It may not be supported by some assemblers (for MASM, the .686 directive must be used). The encoding is provided to facilitate hard-coding of this instruction if it is not supported by the assembler.

This instruction compares the content of ST(0) and ST(*i*), and then sets the ZF, PF and CF flags of the CPU's flag register according to the result of the comparison as follows:

	ZF	PF	CF
If $ST(0) > ST(i)$	0	0	0
If $ST(0) < ST(i)$	0	0	1
If $ST(0) = ST(i)$	1	0	0
If $ST(0) \neq ST(i)$	1	1	1

The condition codes field of the Status Word is not modified.

An Invalid operation exception is detected if either register is empty or contains a [NAN](#), setting the related flag in the [Status Word](#). The result would be indeterminate and return ZF=1, PF=1 and CF=1 as indicated above. (Values of [INFINITY](#) will be treated as valid operands and yield a valid result without any exception being detected.)

A Stack Fault exception is also detected if ST(0) (or the data register specified as the source) is empty, setting the related flag in the Status Word.

Compared to the [FCOM](#) instruction, FCOMI eliminates the need to transfer the Status Word to AX, and then transfer the condition codes from AH to the CPU's flag register prior to conditional jumps. However, FCOMI does not allow comparisons between ST(0) and values in memory.

The following example hard-codes the instruction for comparing ST(0) to ST(2).

```

db 0dbh, 0f0h+2 ;encoding for fcomi st,st(2)
                    ;when not supported by the assembler
fwait            ;insure the instruction is completed
jpe error_handler ;the comparison was indeterminate
                    ;this condition should be verified first
                    ;then only two of the next three conditional jumps
                    ;should become necessary, in whatever order is preferred,
                    ;the third one being replaced by code to handle that case
ja st0_greater   ;when all flags are 0
jb st0_lower     ;only the CF flag would be set if no error
jz both_equal    ;only the ZF flag would be set if no error

```

If these newer instructions are not supported by the assembler, macros could be prepared to hard-code them as above. An example of such a macro would be as follows:

```

fcomist MACRO i
    db 0dbh,0f0h+i
ENDM

```

which could then be used in the above example code as follows:

```
fcomist 2
```

FCOMIP (Compare ST(0) to ST(i) and set CPU flags and Pop ST(0))

Syntax: **fcomip st,st(i)**

Exception flags: Stack Fault, Invalid operation

Encoding: DF F0+i

Note: This instruction is valid only for the Pentium Pro and subsequent processors. It may not be supported by some assemblers (for MASM, the .686 directive must be used). The encoding is provided to facilitate hard-coding of this instruction if it is not supported by the assembler.

This instruction is the same as the FCOMI instruction with the only difference that the top data register is popped after the comparison is completed. This instruction is used when the value in ST(0) would no longer be needed for further computation after the comparison has been performed. See the [FCOMI](#) instruction for other details.

FUCOMI (Unordered Compare ST(0) to ST(i) and set CPU flags)Syntax: **fucomi st,st(i)**

Exception flags: Stack Fault, Invalid operation

Encoding: DB E8+i

Note: This instruction is valid only for the Pentium Pro and subsequent processors. It may not be supported by some assemblers (for MASM, the .686 directive must be used). The encoding is provided to facilitate hard-coding of this instruction if it is not supported by the assembler.

This instruction is the same as the FCOMI instruction with the only difference that the Invalid operation exception will be detected only if one or both operands are an SNAN type of [NAN](#). The QNaNs would not result in an invalid operation but would still set the ZF, PF and CF flags of the CPU's flag register (undefined comparison). See the [FCOMI](#) instruction for other details.

FUCOMIP (Unordered Compare ST(0) to ST(i) and set CPU flags and Pop ST(0))Syntax: **fucomip st,st(i)**

Exception flags: Stack Fault, Invalid operation

Encoding: DF E8+i

Note: This instruction is valid only for the Pentium Pro and subsequent processors. It may not be supported by some assemblers (for MASM, the .686 directive must be used). The encoding is provided to facilitate hard-coding of this instruction if it is not supported by the assembler.

This instruction is the same as the FUCOMI instruction with the only difference that the top data register is popped after the comparison is completed. This instruction is used when the value in ST(0) would no longer be needed for further computation after the comparison has been performed. See the [FUCOMI](#) instruction for other details.

FICOM (Compare ST(0) to an integer in memory)Syntax: **ficom Src**

Exception flags: Stack Fault, Invalid operation, Denormalized value

This instruction performs a signed comparison between the value in the TOP data register ST(0) and the integer value from the specified source (*Src*). The source can be the memory address of either a 16-bit WORD or a 32-bit DWORD integer value (see Chap.2 for [addressing modes of integers](#)). (The [FCOM](#) instruction must be used to compare the value in ST(0) to floating point values).

Note that ST(0) cannot be compared to a 64-bit QWORD integer in memory. If that becomes necessary, the QWORD integer value must first be loaded to the FPU and the FCOM or [FCOMP](#) instruction used.

The values of ST(0) and of the source are not modified by this instruction.

The result of the comparison is reported in the condition codes field of the [Status Word](#) as follows (the C1 bit is not used and the C2 bit was not used in early FPUs):

	C3	C2	C0
If ST(0) > source	0	0	0
If ST(0) < source	0	0	1
If ST(0) = source	1	0	0
If ST(0) ? source	1	1	1

An **Invalid operation** exception is detected if ST(0) is empty, or if its value is a [NAN](#), setting the related flag in the Status Word. The result would be indeterminate and return C3=1, C2=1 and C0=1 as indicated above. (Values of [INFINITY](#) in ST(0) will be treated as valid operands and yield a valid result without any exception being detected.)

A **Stack Fault** exception is also detected if ST(0) is empty, setting the related flag in the Status Word.

When the value in ST(0) is a [denormalized](#) number, a **Denormal** exception is detected, setting the related flag in the Status Word. The comparison would still yield a valid result.

The signed comparison means that positive values are larger than negative values, including the values of INFINITY which can be positive or negative. The lowest of all valid values in ST(0) would thus be - INFINITY and the greatest would be +INFINITY when used in comparisons.

When the Status Word containing the result of the comparison is stored in AX, the C3, C2 and C0 bits can be transferred to the CPU's flag register (using the **sahf** instruction) as the ZF, PF and CF flags respectively. This makes it easier for branching based on the result. Following would be typical code after a comparison with an integer in memory. This example makes use of the possibility of pushing immediate values on the stack.

```
pushd 125          ;push the immediate value as a DWORD
ficom dword ptr[esp] ;compare ST(0) to the integer value on the stack
fstsw [esp]        ;overwrite the pushed value with the Status Word
fwait              ;insure the previous instruction is completed
pop eax             ;retrieve the Status Word in AX and clean the stack
sahf                ;transfer the condition codes to the CPU's flag register
jpe error_handler  ;the comparison was indeterminate
                   ;this condition should be verified first
                   ;then only two of the next three conditional jumps
                   ;should become necessary, in whatever order is preferred,
                   ;the third jump being replaced by code to handle that case
ja    st0_greater  ;when all flags are 0
jb    st0_lower    ;only the C0 bit (CF flag) would be set if no error
jz    both_equal   ;only the C3 bit (ZF flag) would be set if no error
```

FICOMP (Compare ST(0) to an integer in memory and POP ST(0))

Syntax: **ficomp Src**

Exception flags: Stack Fault, Invalid operation, Denormalized value

This instruction is the same as the FICOM instruction with the only difference that the top data register is popped after the comparison is completed. This instruction is used when the value in ST(0) would no longer be needed for further computation after the comparison has been performed. See the [FICOM](#) instruction for other details. (The [FCOMP](#) instruction must be used to compare the value in ST(0) to a floating point value and POP the TOP data register).

The popped value is not stored anywhere. The FPU simply modifies the tag of the current TOP register to "empty" in the [Tag Word](#), and increments the TOP field in the [Status Word](#).

FTST (Test ST(0) by comparing it to +0.0)

Syntax: **ftst** (no operand)

Exception flags: Stack Fault, Invalid operation, Denormalized value

This instruction performs a signed comparison between the value in the TOP data register ST(0) and +0.0. The value of ST(0) is not modified by this instruction.

The result of the comparison is reported in the condition codes field of the [Status Word](#) as follows (the C1 bit is not used and the C2 bit was not used in early FPUs):

	C3	C2	C0
If ST(0) > 0	0	0	0
If ST(0) < 0	0	0	1
If ST(0) = 0	1	0	0
If ST(0) ? 0	1	1	1

An **Invalid operation** exception is detected if ST(0) is empty, or if its value is a [NAN](#), setting the related flag in the Status Word. The result would be indeterminate and return C3=1, C2=1 and C0=1 as indicated above. (Values of [INFINITY](#) will be treated as valid operands and yield a valid result without any exception being detected.)

A **Stack Fault** exception is also detected if ST(0) is empty, setting the related flag in the Status Word.

When the value of ST(0) is a [denormalized](#) number, a **Denormal** exception is detected, setting the related flag in the Status Word. The comparison would still yield a valid result.

A value of -0.0 in ST(0) would be treated as equal to a +0.0 value.

This instruction is used primarily to determine if the value in ST(0) is positive, or negative, or equal to zero, if it is a valid floating point value. For example, such a test should be performed before attempting to compute the logarithm of a number or extract its square root. Following would be typical code with this instruction.

```

ftst           ;compare the value of ST(0) to +0.0
fstsw ax       ;copy the Status Word containing the result to AX
fwait          ;insure the previous instruction is completed
sahf           ;transfer the condition codes to the CPU's flag register
jpe error_handler ;the comparison was indeterminate
                  ;this condition should be verified first
                  ;then only two of the next three conditional jumps
                  ;should become necessary, in whatever order is preferred,
                  ;the third jump being replaced by code to handle that case
ja st0_positive ;when all flags are 0
jb st0_negative ;only the C0 bit (CF flag) would be set if no error
jz st0_zero     ;only the C3 bit (ZF flag) would be set if no error

```

FXAM (Examine the content of ST(0))

Syntax: **fxam** (no operand)

Exception flags: None

This instruction examines the content of ST(0) and the type of value is reported in the condition codes C3, C2 and C0 of the [Status Word](#) as indicated below. In addition, the C1 bit would be the same as the sign bit of the value in ST(0).

Content of ST(0)	C3	C2	C0
Unsupported	0	0	0
NAN	0	0	1
Normal finite number	0	1	0
Infinity	0	1	1
Zero	1	0	0
Empty	1	0	1
Denormalized number	1	1	0

The following example code would examine the content of a temporary variable to determine the action required by the ensuing code according to the result. The condition codes are transferred to the CPU's flag register for the main conditional jumps (C3=ZF, C2=PF, C0=CF). The action required for each case will then depend on the purpose of the examination and is left blank. In most cases, only a few of the cases may require special action.

```

fld temp_var ;load the temporary variable to the FPU
fxam          ;examine it
fstsw ax      ;copy the content of the Status Word to AX

```

```

fwait          ;insure the last instruction is completed
sahf           ;copy the C3/C2/C0 condition codes to the ZF/PF/CF flags
jz    C3is1    ;either Zero, Empty or Denormalized if C3=1
jpe    C2is1    ;either normal or infinity if C3=0 and C2=1
jc    isNAN     ;would be NAN if C3=0, C2=0 and C0=1
.....         ;code for the case of Unsupported, no need to check sign

```

```

isNAN:
.....         ;code for the case of a NAN, no need to check the sign

```

```

C2is1:
jc    isINFINITY ;would be Infinity if C3=0, C2=1 and C0=1
                ;this leaves the case for a Normal finite number
test ah,2       ;test for the sign which is in bit1 of AH
jnz    negNORMAL
.....         ;code for the case of a positive Normal finite number

```

```

negNORMAL:
.....         ;code for the case of a negative Normal finite number

```

```

isINFINITY:
test ah,2       ;test for the sign which is in bit1 of AH
jnz    negINFINITY
.....         ;code for the case of a positive Infinity

```

```

negINFINITY:
.....         ;code for the case of a negative Infinity

```

```

C3is1:
jc    isEMPTY   ;would be Empty if C3=1 and C0=1
jpe    isDENORMAL ;would be a Denormalized number if C3=1, C0=0 and C2=1
                ;this leaves the case for a Zero value
.....         ;code for the case of a Zero value, no need to check sign

```

```

isEMPTY:
.....         ;code for the case of an Empty register
                ;which does not apply in this example because
                ;ST(0) was loaded with a value from memory

```

```

isDENORMAL:
test ah,2       ;test for the sign which is in bit1 of AH
jnz    negDENORMAL
.....         ;code for the case of a positive Denormalized number

```

```

negDENORMAL:
.....         ;code for the case of a negative Denormalized number

```

[RETURN TO](#)
[SIMPLY FPU](#)
[CONTENTS](#)